

第3章 栈和队列

主讲教师：黄绍辉



栈和队列是两种应用非常广泛的数据结构，它们都来自线性表数据结构，都是“**操作受限**”的线性表。

栈在计算机的实现有多种方式：

- ◆ **硬堆栈**：利用CPU中的某些寄存器组或类似的硬件或使用内存的特殊区域来实现。这类堆栈容量有限，速度很快；
- ◆ **软堆栈**：这类堆栈主要在内存中实现。堆栈容量可以达到很大。在实现方式上，又有**动态方式**和**静态方式**两种。

本章将讨论栈和队列的基本概念、存储结构、基本操作以及这些操作的具体实现。



3.1 栈

1 栈的概念

栈(Stack)：是限制在表的一端进行插入和删除操作的线性表。又称为后进先出**LIFO (Last In First Out)**或先进后出**FILO (First In Last Out)**线性表。

栈顶(Top)：允许进行插入、删除操作的一端，又称为表尾。用栈顶指针(top)来指示栈顶元素。

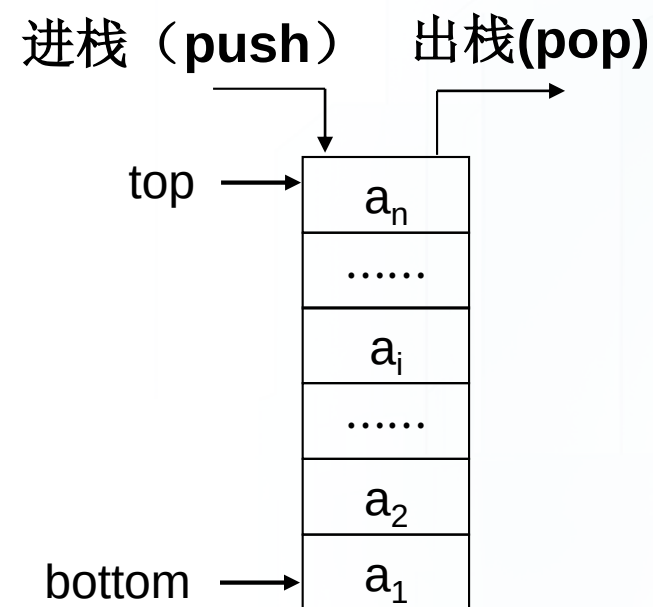
栈底(Bottom)：是固定端，又称为表头。

空栈：当表中没有元素时称为空栈。



设栈 $S=(a_1, a_2, \dots, a_n)$ ，则 a_1 称为栈底元素， a_n 为栈顶元素，如图所示。

栈中元素按 $a_1, a_2, \dots, a_n$ 的次序进栈，退栈的第一个元素应为栈顶元素。即栈的修改是按后进先出的原则进行的。



顺序栈示意图

2 栈的抽象数据类型定义

ADT Stack{

数据对象: $D = \{ a_i | a_i \in \text{ElemSet}, i=1,2,\dots,n, n \geq 0 \}$

数据关系: $R = \{ \langle a_{i-1}, a_i \rangle | a_{i-1}, a_i \in D, i=2,3,\dots,n \}$

基本操作: 初始化、进栈、出栈、取栈顶元素等

} ADT Stack

栈的顺序存储表示

栈的顺序存储结构简称为顺序栈，和线性表相类似，用**一维数组**来**存储栈**。根据数组是否可以根据需要增大，又可分为**静态顺序栈**和**动态顺序栈**。

- ◆ **静态顺序栈**实现简单，但不能根据需要增大栈的存储空间；
- ◆ **动态顺序栈**可以根据需要增大栈的存储空间，但实现稍为复杂。



栈的动态顺序存储表示

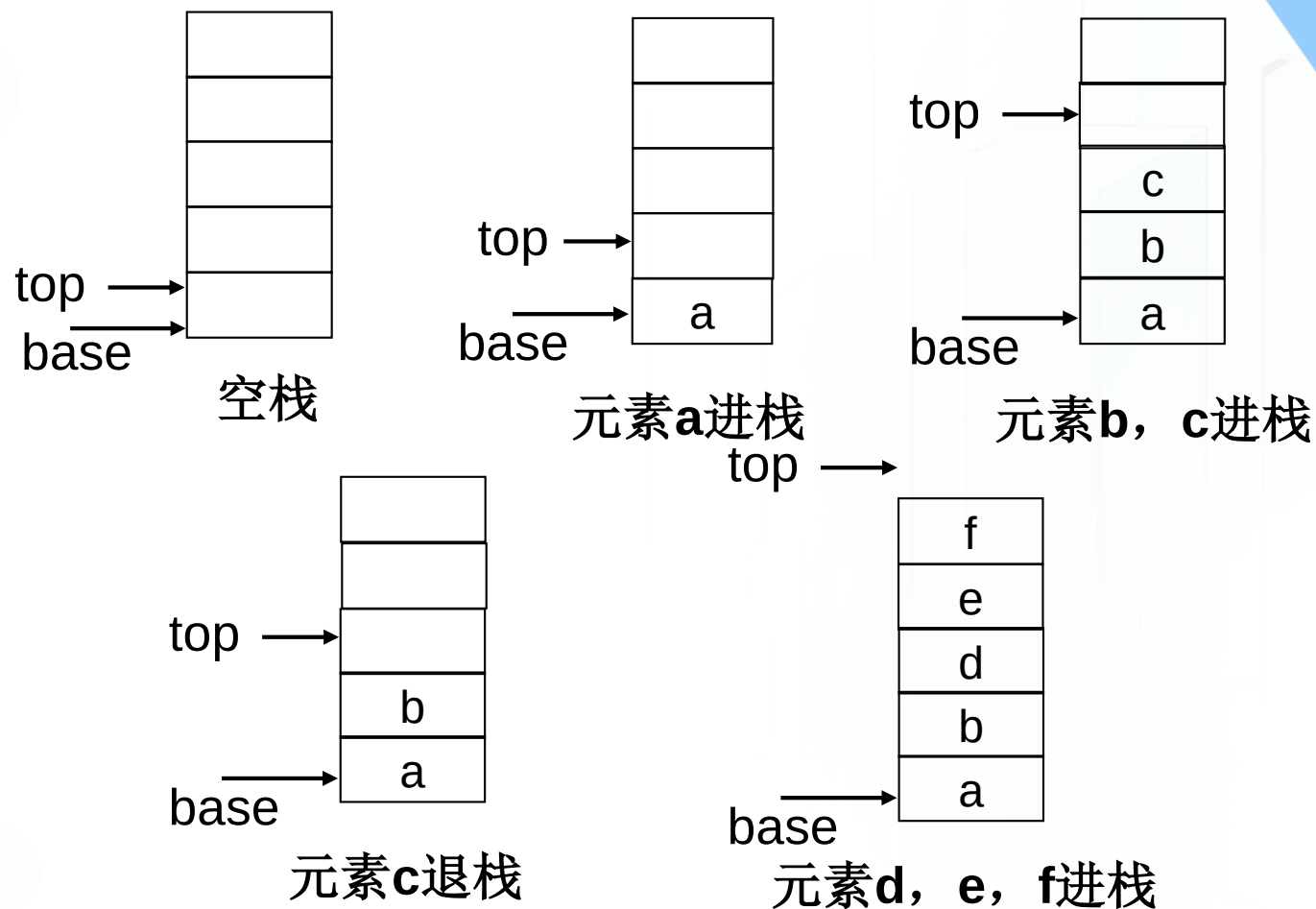
采用**动态一维数组**来**存储栈**。所谓动态，指的是栈的大小可以根据需要增加。

- ◆ 用base表示栈底指针，栈底固定不变的；栈顶则随着进栈和退栈操作而变化。用top(称为栈顶指针)指示当前栈顶位置。
- ◆ 用 $top = base$ 作为栈空的标记，每次top指向栈顶数组中的下一个存储位置。



✓ **结点进栈**：首先将数据元素保存到栈顶(**top**所指的当前位置)，然后执行**top**加1，使**top**指向栈顶的下一个存储位置；

✓ **结点出栈**：首先执行**top**减1，使**top**指向栈顶元素的存储位置，然后将栈顶元素取出。



(动态)堆栈变化示意图

基本操作的实现

1 栈的类型定义

```
#define STACK_INIT_SIZE 100  /* 栈初始向量大小 */
#define STACKINCREMENT 10  /* 存储空间分配增量 */
typedef struct {
    SElemType *base;  /* 栈不存在时值为NULL */
    SElemType *top;    /* 栈顶指针 */
    int  stacksize ;   /* 当前可使用的最大空间 */
} SqStack ;
```

2 栈的初始化

```
Status InitStack(SqStack &S) {  
    S.base=(SElemType *)malloc(STACK_INIT_SIZE *  
                                sizeof(SElemType));  
  
    if (! S.base) return  ERROR;  
    S.top=S.base ;   /* 栈空时栈顶和栈底指针相同 */  
    S.stacksize=STACK_INIT_SIZE;  
    return OK ;  
} //InitStack
```

3 压栈(元素进栈)

```
Status push(SqStack &S, SElemType e) {  
    if (S.top-S.base>=S.stacksize) {  
        S.base=(SElemType *)realloc((S.base,  
        (S.stacksize+STACKINCREMENT)*sizeof(SElemType));  
        if (! S.base) return ERROR;  
        S.top=S.base+S.stacksize ;  
        S.stacksize+=STACKINCREMENT ;  
    }  
    *S.top++=e; return OK;  
} //push
```

4 弹栈(元素出栈)

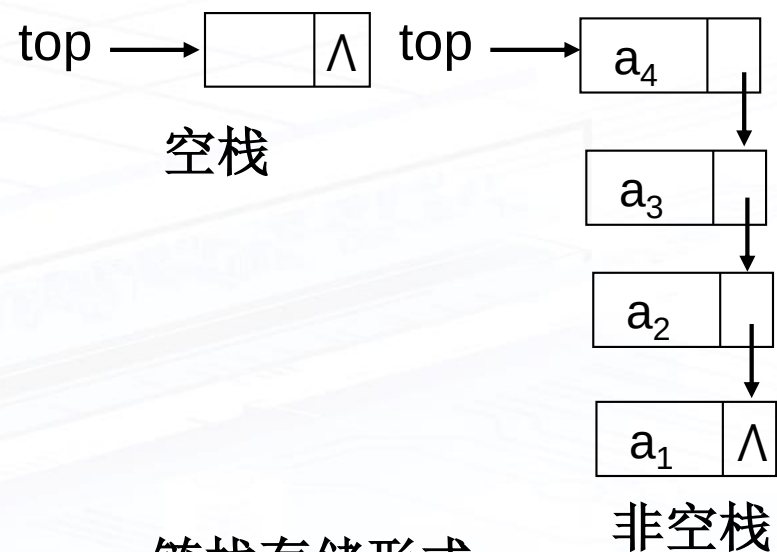
```
Status pop( SqStack &S, SElemType &e) {  
    if ( S.top==S.base )  
        return ERROR;    /* 栈空, 返回失败标志 */  
    e=*&S.top;   
    return OK;   
} //pop
```

栈的链式存储表示

栈的**链式存储结构**称为链栈，是运算受限的单链表。其插入和删除操作只能在表头位置上进行。因此，链栈没有必要像单链表那样附加头结点，栈顶指针top就是链表的头指针。图为栈的链式存储表示形式。

链栈的结点类型说明如下：

```
typedef struct Stack_Node
{ ElemType data ;
  struct Stack_Node *next ;
} Stack_Node ;
```



链栈存储形式



3.2 栈的应用举例

由于栈具有的**后进先出**的固有特性，因此，栈成为程序设计中常用的工具和数据结构。以下是几个栈应用的例子。



3.2.1 数制转换

十进制整数 n 向其它进制数 d (二、八、十六)的转换是计算机实现计算的基本问题。

转换法则：该转换法则对应于一个简单算法原理：

$$n = (n \text{ div } d) * d + n \text{ mod } d$$

其中： div 为整除运算, mod 为求余运算。

注意到余数是**倒序**输出的，也就是**后进先出**。

例如 $(1348)_{10} = (2504)_8$ ，其运算过程如下：

n	$n \text{ div } 8$	$n \text{ mod } 8$
1348	168	4
168	21	0
21	2	5
2	0	2

算法3.1 , 十进制转八进制

```
void conversion( ) {  
    Init_Stack(S);  
    scanf("%d", &n);  
    while (n) {  
        push(S, n%8); n=n/8 ;  
    }  
    while (!StackEmpty(S)) {  
        pop(S, e); printf("%d" , e) ;  
    }  
} //conversion
```

3.2.2 括号匹配问题

在文字处理软件或编译程序设计时，常常需要检查一个字符串或一个表达式中的括号是否相匹配？

匹配思想：从左至右扫描一个字符串(或表达式)，**将每个右括号与最近遇到的那个左括号相匹配**。可以在从左至右扫描过程中把所遇到的左括号存放到堆栈中。每当遇到一个右括号时，就将它与栈顶的左括号(如果存在)相匹配，同时从栈顶删除该左括号。

算法思想：设置一个栈，当读到左括号时，左括号进栈。当读到右括号时，则从栈中弹出一个元素，与读到的左括号进行匹配，若匹配成功，继续读入；否则匹配失败，返回FALSE。

3.2.3 行编辑程序

行编辑程序能接收用户的一行输入，由于光标不能回退，因此需要支持特殊字符#和@，统一处理后再存入用户数据区。例如：

用户输入	处理后的结果
whli##ilr#e(s#*s) outcha@putchar(*s=#++);	while(*s) putchar(*s++);

思路：逐行处理。从左至右扫描一个字符串，遇到#则删除前一个字符，遇到@则删除之前的所有字符。

算法思想：设置一个栈，当读到普通字符时直接进栈；当读到#时，则从栈中弹出一个元素；当读到@时，则清空栈。字符读取完毕后，将整个栈的内容传送到用户数据区。

3.2.4 迷宫求解

思路：(1) 判断当前探索位置`curpos`（最开始是起点）是否可通，是则前进一步，并生成下一个探索位置；否则，回退一步，尝试生成下一个探索方向。

(2) 回退后，如果该位置`di=4`，就标记该位置**没前途**（防止后续再次进入同一位置），然后继续回退直到**`di<4`或者栈空**。如果**`di<4`**，就生成下一个探索位置，转回(1)继续判断；如**栈空**则无解。

前进一步就是入栈，回退一步就是出栈。

`di`取值1~4分别表示东、南、西、北四个方向（方向的顺序无所谓），探索顺序是从1到4。如果某个位置的`di`为4，表示4个方向都已探索完毕。

算法3.3 , 迷宫求解

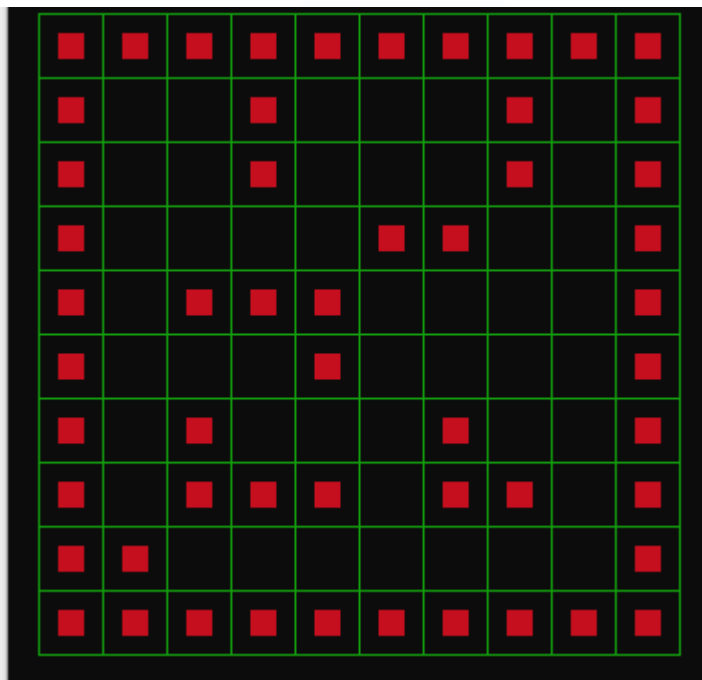
```
Status MazePath(MazeType maze, PosType start, PosType end) {  
    InitStack(S); curpos=start; curstep=1;  
    do {  
        if (Pass(curpos)) { //即将走入的位置可以通过  
            FootPrint(curpos);  
            e=(curstep, curpos, 1);  
            Push(S, e); //入栈是为了保存最近一步的位置和方向，方便回退  
            if (curpos==end) return TRUE;  
            curpos=NextPos(curpos, 1); //下一探索位置，注意还没走过去  
            curstep++;  
        } //if
```

```
else { //当前位置不可通过
    if (!StackEmpty(S)) {
        Pop(S, e); //出栈，尝试换方向
        while (e.di==4 && !StackEmpty(S)) { //无方向可换，必须回退
            MarkPrint(e.seat); Pop(S, e); curstep--;
        } //while
        if (e.di<4) { //如果还有方向可以尝试，入栈
            e.di++; Push(S, e); curpos=NextPos(e.seat, e.di);
        } //if
    } //if
} //else
} while (!StackEmpty(S));
return FALSE;
} //MazePath
```

步数减一，课本漏了这个会导致步数算错。

这个算法属于深度优先遍历DFS，找到的路径通常不是最短的；如果要找最短路径，需要用队列来实现广度优先搜索BFS。

走迷宫示意



从迷宫的前几步理解一下算法:

1. 尝试走入初始位置(1,1)，可通过，于是((1,1),1)入栈，下一位置(1,2)；
2. 尝试走入位置(1,2)，可通过，于是((1,2),1)入栈，下一位置(1,3)；
3. 尝试走入位置(1,3)，不可通过，于是((1,2),1)出栈，下一方向((1,2),2)入栈，下一位置(2,2)；
4. 尝试走入位置(2,2)，可通过，于是((2,2),1)入栈，下一位置(2,3)；
5. 尝试走入位置(2,3)，不可通过，于是((2,2),1)出栈，下一方向((2,2),2)入栈，下一位置(3,2)；
-

```
typedef struct {  
    .....  
    int ord; /*序号*/ PosType seat; /*位置*/ int di; /*方向*/  
} SElemType;
```

3.2.5 表达式求值

算法基本思想：使用两个工作栈，一个是OPTR，用以存放运算符，另一个是OPND，用于存数。

1. 首先置OPND为空栈，OPTR存放#作为栈底元素；
2. 依次读入表达式的每个字符，
 - (1) 若是操作数，入栈OPND；
 - (2) 若是运算符，与OPTR的栈顶运算符比较优先级大小，然后根据比较结果执行相应的操作，直到整个表达式求值完毕（OPTR的栈顶和栈底元素都是#）。

比较运算符之间的优先关系，可以通过查表实现，参考表3.1。

算法3.4 , 表达式求值

```
OperandType EvaluateExpression() {  
    InitStack(OPTR); Push(OPTR, '#'); //运算符栈先压一个#  
    InitStack(OPND); c=getchar();  
    while (c!='#' || GetTop(OPTR)!='#') {  
        if (!In(c, OP)) { Push(OPND, c); c=getchar(); } //操作数直接入栈  
        else {  
            switch (Precede(GetTop(OPTR), c)) {  
                case '<': //栈顶元素优先级低  
                    Push(OPTR, c); c=getchar();  
                    break;
```

case '=': //脱括号并接收下一字符

Pop(OPTR, x); c=getchar();

break;

case '>': //退栈并将运算结果入栈

Pop(OPTR, theta);

Pop(OPND, b); Pop(OPND, a);

Push(OPND, Operate(b, theta, a);

break;

} //switch

} //while

return GetTop(OPND);

} //EvaluateExpression

表达式求值示例：3*(7-2)

步骤	OPTR栈 θ_1	OPND栈	输入字符 θ_2	主要操作
1	#		<u>3</u> *(7-2)#	Push(OPND, '3')
2	#	3	*(<u>7</u> -2)*	Push(OPTR, '*')
3	# *	3	(<u>7</u> -2)#	Push(OPTR, '(')
4	# * (	3	<u>7</u> -2)#	Push(OPND, '7')
5	# * (	3 7	<u>-</u> 2)#	Push(OPTR, '-')
6	# * (-	3 7	<u>2</u>)#	Push(OPND, '2')
7	# * (-	3 7 2	)#	operate('7', '-', '2'); Push(OPND, '5')
8	# * (	3 5	)#	Pop(OPTR) 消去一对括号
9	# *	3 5	#	operate('5', '*', '3'); Push(OPND, '15')
10	#	15	#	return GetTop(OPND);

3.3 栈与递归的实现

栈的另一个重要应用是在程序设计语言中实现递归调用。

递归调用：一个函数(或过程)直接或间接地调用自己本身，简称**递归**(Recursive)。

递归是程序设计中的一个强有力的工具。因为递归函数结构清晰，程序易读，正确性很容易得到证明。

为了使递归调用不至于无终止地进行下去，实际上有效的递归调用函数(或过程)应包括两部分：**递推规则(方法)**，**终止条件**。



例：求n!

$$\text{fact}(n) = \begin{cases} 1 & \text{当 } n=0 \text{ 时} & \text{终止条件} \\ n * \text{fact}(n-1) & \text{当 } n > 0 \text{ 时} & \text{递推规则} \end{cases}$$

为保证递归调用正确执行，系统设立一个“**递归工作栈**”，作为整个递归调用过程期间使用的数据存储区。

每一层递归包含的信息如：**参数**、**局部变量**、**上一层的返回地址**构成一个“**工作记录**”。每进入一层递归，就产生一个新的**工作记录**压入栈顶；每退出一层递归，就从栈顶弹出一个**工作记录**。

◆ 从被调函数返回调用函数的一般步骤：

- ① 若栈为空，则执行正常返回。
- ② 从栈顶弹出一个工作记录。
- ③ 将“工作记录”中的参数值、局部变量值赋给相应的变量；
读取返回地址。
- ④ 将函数值赋给相应的变量。
- ⑤ 转移到返回地址。

◆ 函数调用和递归调用的最大区别，就是后者的入口地址始终是自己。如果编程语言不支持递归，那就需要仿照上述的步骤，自行维护一个递归工作栈来模拟递归。下面将用fact函数举例。

阶乘的递归模拟

```
#include <iostream>
#include <stack>
using namespace std;
int fact1(int n);
int fact2(int n);
int main()
{
    printf("%d %d\n", fact1(6), fact2(6));
    return 0;
}
int fact1(int n)
{
    if (n==0) return 1;
    else return n*fact1(n-1);
}
```

递归模拟的核心思想：

- 用栈非空的循环替代递归调用；
- 每次循环先出栈一次，模拟递归返回；
- 递归调用改为入栈。

```
int fact2(int n)
{
    stack<int> S;
    int f=1;
    S.push(n);
    while (!S.empty()) {
        int curr=S.top();
        S.pop();
        if (curr==0)
            f *= 1;
        else { //递归调用模拟
            f *= curr;
            S.push(curr-1);
        }
    }
    return f;
}
```

3.4 队列

1 队列的基本概念

队列(Queue)：也是运算受限的线性表。是一种**先进先出**(First In First Out，简称**FIFO**)的线性表。只允许在表的一端进行插入，而在另一端进行删除。

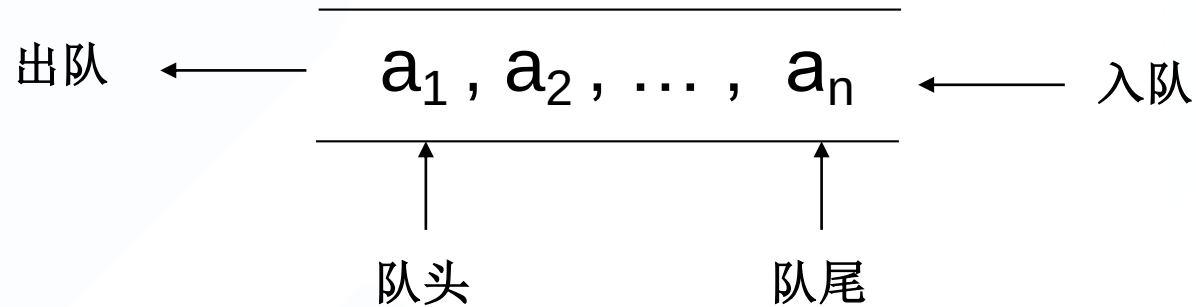
队首(front)：允许进行删除的一端称为队首。

队尾(rear)：允许进行插入的一端称为队尾。

例如：排队购物。操作系统中的作业排队。先进入队列的成员总是先离开队列。



队列中没有元素时称为空队列。在空队列中依次加入元素 $a_1, a_2, \dots, a_n$ 之后， a_1 是队首元素， a_n 是队尾元素。显然退出队列的次序也只能是 $a_1, a_2, \dots, a_n$ ，即队列的修改是依**先进先出**的原则进行的，如图所示。



队列示意图

2 队列的抽象数据类型定义

ADT Queue{

数据对象: $D = \{ a_i | a_i \in \text{ElemSet}, i=1, 2, \dots, n, n \geq 0 \}$

数据关系: $R = \{ \langle a_{i-1}, a_i \rangle \mid a_{i-1}, a_i \in D, i=2,3,\dots,n \}$

约定 a_1 端为队首, a_n 端为队尾。

基本操作:

InitQueue(&Q): 创建一个空队列Q;

DestroyQueue(&Q): 销毁队列Q;

.....

EnQueue(&Q, e) : 向Q的队尾插入元素e;

DeQueue(&Q, &e) : 删除Q的队头元素e;

.....

} ADT Queue

3.4.2 队列的链式表示和实现

队列的链式存储结构简称为链队列，它是限制仅在表头进行删除操作和表尾进行插入操作的单链表。

需要两类不同的结点：**数据元素**结点，队列的**队头指针**和**队尾指针**的结点。

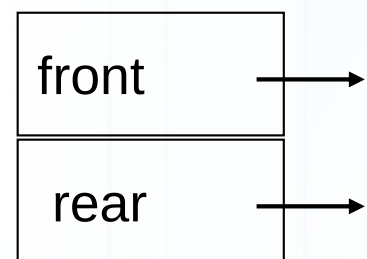


数据元素结点类型定义：

```
typedef struct QNode {  
    QElemType  data ;  
    struct QNode *next ;  
} QNode, *QueuePtr;
```



数据元素结点



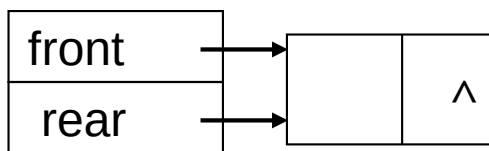
指针结点

指针结点类型定义：

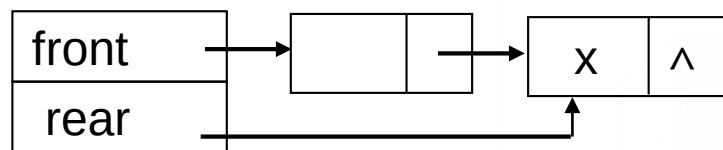
```
typedef struct {  
    QueuePtr front ;  
    QueuePtr rear ;  
} LinkQueue ;
```

链队列结点示意图

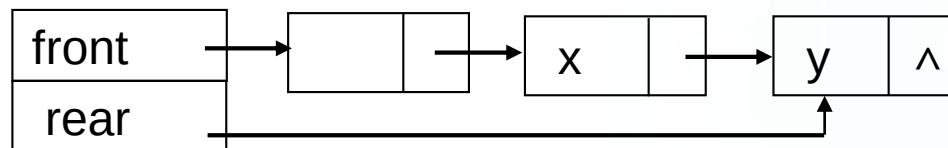
- ◆ 使用带头结点的链表。
- ◆ 链队的操作实际上是单链表的操作，只不过是删除在表头进行，插入在表尾进行。
- ◆ 插入、删除时分别修改不同的指针。链队运算及指针变化如图所示。



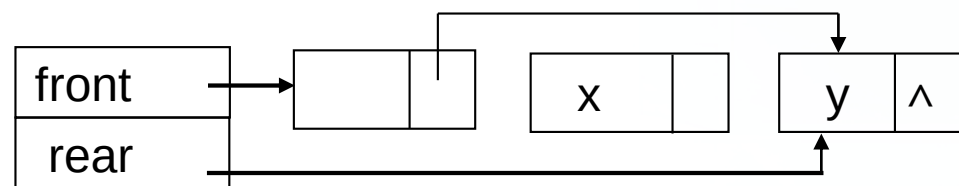
(a) 空队列



(b) x入队



(c) y再入队



(d) x出队

3 链队列的基本操作

(1) 链队列的初始化

```
Status InitQueue(LinkQueue &Q) {  
    Q.front=Q.rear=(QueuePtr) malloc(sizeof(QNode)) ;  
    if (!Q.front) exit(OVERFLOW);  
    Q.front->next=NULL;  
    return OK;  
}
```

(2) 链队列的入队操作

在已知队列的队尾插入一个元素 e ，即修改队尾指针($Q.rear$)。

```
Status EnQueue(LinkQueue &Q, QElemType e) {  
    p=(QueuePtr)malloc(sizeof(QNode));  
    if (!p) exit(OVERFLOW);  
    p->data=e ; p->next=NULL ;  
    Q.rear->next=p ;  
    Q.rear=p ;  
    return OK;  
}
```

(3) 链队列的出队操作

```
Status DeQueue(LinkQueue &Q, QElemType &e) {  
    if (Q.front==Q.rear) return ERROR ;  
    p=Q.front->next ;  
    e=p->data ;  
    Q.front->next=p->next ;  
    if (Q.rear==p) Q.rear=Q.front ;  
    free(p) ;  
    return OK ;  
}
```

(4) 链队列的撤消

```
Status DestroyQueue(LinkQueue &Q) {  
    while (Q.front) {  
        Q.rear=Q.front->next;  
        free(Q.front);  
        Q.front=Q.rear;  
    }  
    return OK ;  
}
```

3.4.3 队列的顺序表示和实现

利用一组连续的存储单元(一维数组)依次存放从队头到队尾的各个元素，称为顺序队列。

结点定义如下：

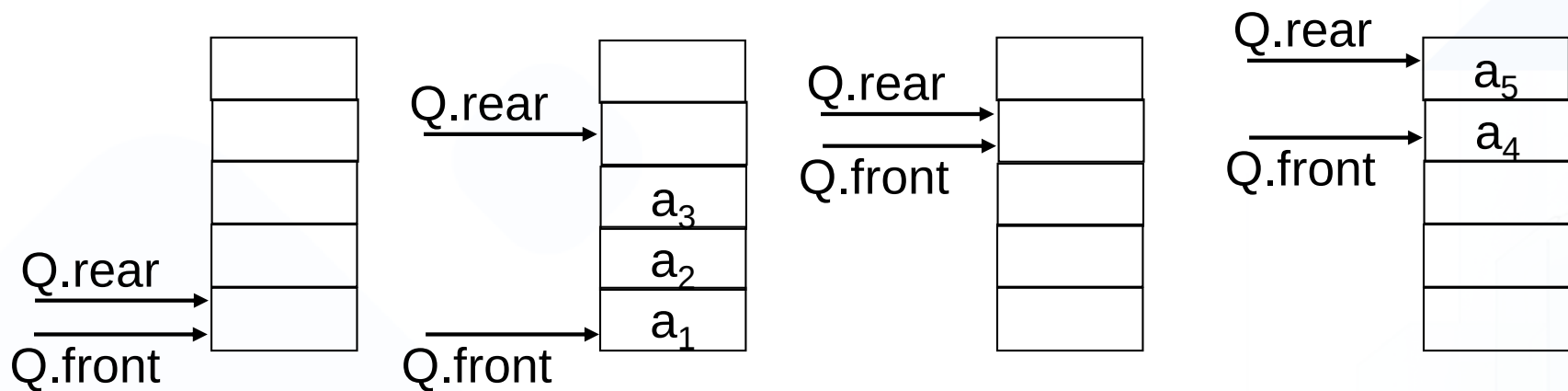
```
#define MAXQSIZE 100
typedef struct {
    QElemType *base;
    int front;
    int rear;
} SqQueue;
```



设立一个队首指针front，一个队尾指针rear，分别指向队首和队尾元素。

- ◆ 初始化：front=rear=0。
- ◆ 入队：将新元素插入rear所指的位置，然后rear加1。
- ◆ 出队：删去front所指的元素，然后加1并返回被删元素。
- ◆ 队列为空：front=rear。
- ◆ 队满：rear=MAX_QUEUE_SIZE-1或front=rear。

在非空队列里，队首指针始终指向队头元素，而队尾指针始终指向队尾元素的下一位置。



(a) 空队列 (b) 入队3个元素 (c) 出队3个元素 (d) 入队2个元素

- ◆ 顺序队列中存在“**假溢出**”现象。因为在入队和出队操作中，头、尾指针只增加不减小，致使被删除元素的空间永远无法重新利用。
- ◆ 尽管队列中实际元素个数可能远远小于数组大小，但可能由于尾指针已超出向量空间的上界而不能做入队操作。该现象称为**假溢出**。如图所示是数组大小为5的顺序队列中队首、队尾指针和队列中元素的变化情况。

3.4.3 循环队列

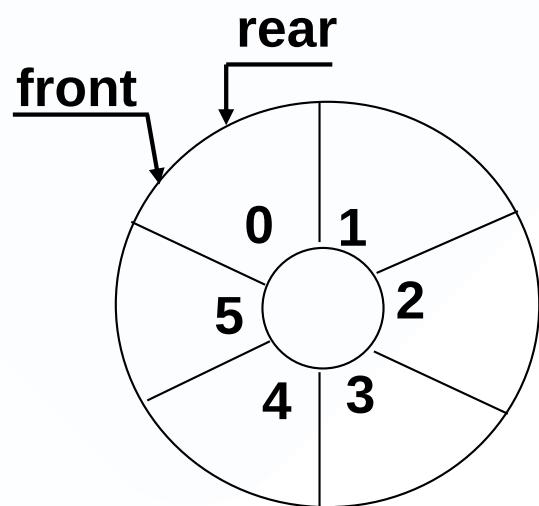
为充分利用向量空间，克服上述“假溢出”现象的方法是：将为队列分配的向量空间看成为一个首尾相接的圆环，并称这种队列为循环队列(Circular Queue)。

在循环队列中进行出队、入队操作时，队首、队尾指针仍要加1，朝前移动。只不过加1之后要对空间大小取模，避免下标溢出，也就是相当于 $(Q.front+1) \% MAXQSIZE$ 或者 $(Q.rear+1) \% MAXQSIZE$

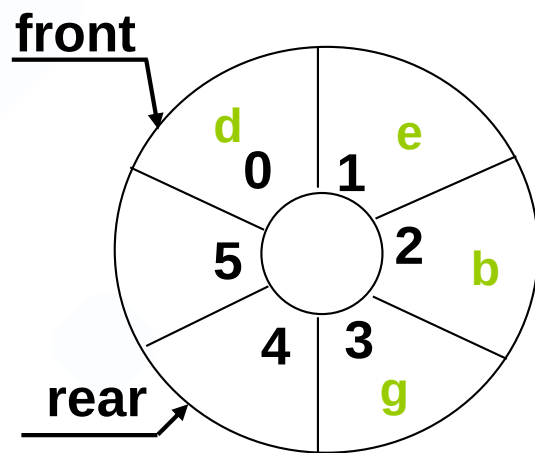


- ◆ 循环队列所分配的空间可以被充分利用，除非向量空间真的被队列元素全部占用，否则不会上溢。因此，真正实用的顺序队列是循环队列。

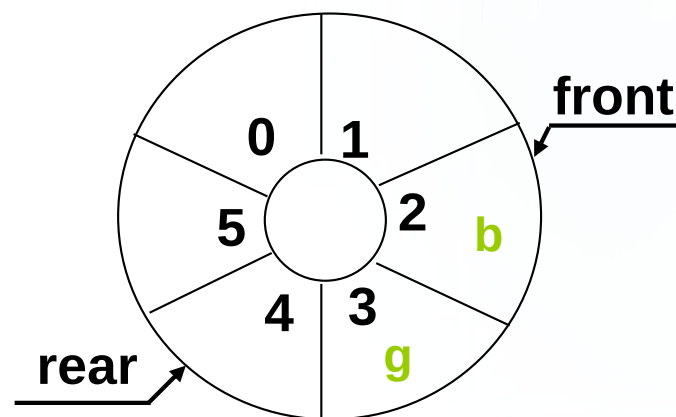
例：设有循环队列 $Q[0, 5]$ ，其初始状态是 $\text{front}=\text{rear}=0$ ，各种操作后队列的头、尾指针的状态变化情况如下图所示。



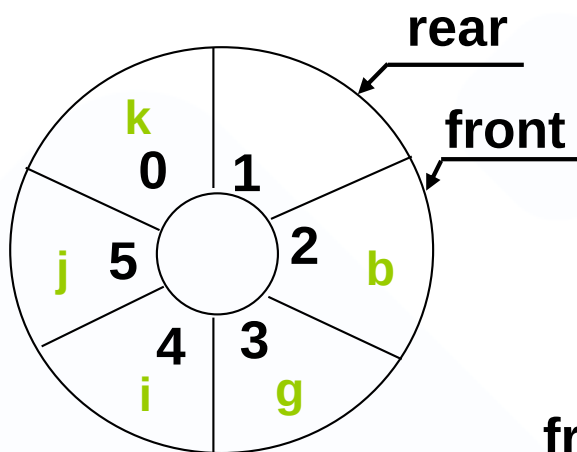
(a) 空队列



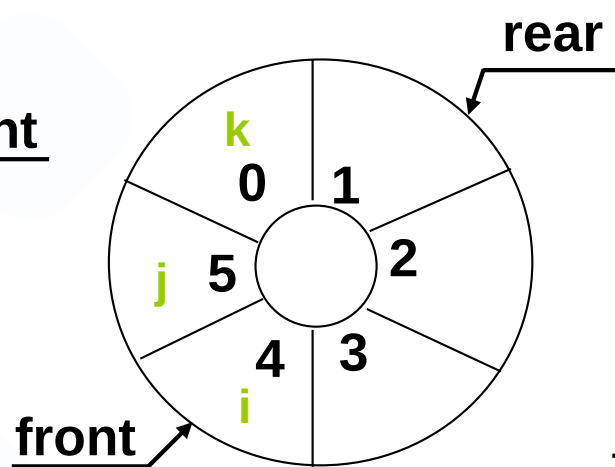
(b) d, e, b, g入队



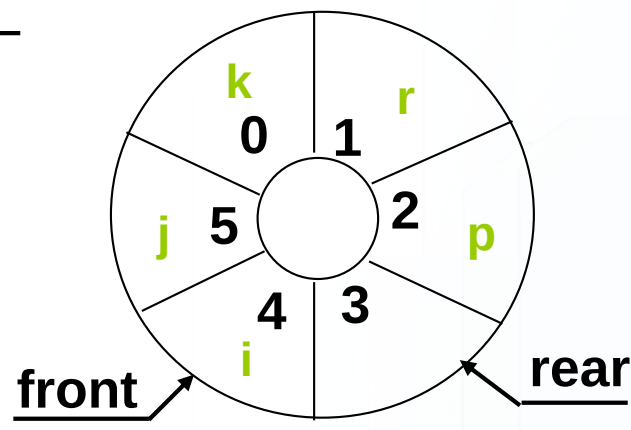
(c) d, e出队



(d) i, j, k入队



(e) b, g出队



(f) r, p, s, t入队

循环队列操作及指针变化情况

入队时尾指针向前追赶头指针，出队时头指针向前追赶尾指针，故队空和队满时头尾指针均相等。因此，无法通过 $front=rear$ 来判断队列“空”还是“满”。

解决队满的判断方法有两个：

- ① 另设一个标志位以区别队列是空还是满；
- ② 约定入队前，测试尾指针在循环意义下加1后是否等于头指针，若相等则认为队满。即：
 - ◆ rear所指的单元始终为空（特意空出该位置不存储数据）。
 - ◆ 循环队列为空： $\text{front} == \text{rear}$
 - ◆ 循环队列满： $(\text{rear} + 1) \% \text{MAXQSIZE} == \text{front}$

循环队列的几个关键函数

1 循环队列的初始化

```
Status InitQueue(SqQueue &Q) {  
    Q.base=(QElemType*)malloc(MAXQSIZE*sizeof(QElemType)) ;  
    if (!Q.base) exit(OVERFLOW);  
    Q.front=Q.rear=0;  
    return OK;  
}
```

2 入队操作

```
Status EnQueue(SqQueue &Q, QElemType e) {  
    if ((Q.rear+1)%MAXQSIZE== Q.front)  
        return ERROR; //队满, 返回错误标志  
    Q.base[Q.rear]=e ;  
    Q.rear=(Q.rear+1)%MAXQSIZE ;  
    return OK;  
}
```

3 出队操作

```
Status DeQueue(SqQueue &Q , QElemType &e) {  
    if (Q.front== Q.rear)  
        return ERROR ;    // 队空 , 返回错误标志  
    e=Q.base[Q.front] ;  
    Q.front=(Q.front+1)%MAXQSIZE ;  
    return OK ;  
}
```

结束

谢谢

